

Axis Robotic Arm Simulator Using Inverse Kinematics

Abstract

This project focuses on the design and simulation of a 6-axis robotic arm capable of moving its end effector from one position to another using inverse kinematics. The robotic arm is simulated in a virtual environment using Python and robotics simulation tools. The system calculates the required joint angles for a given target position and generates a smooth path for the robotic arm. The project demonstrates the fundamentals of robotic manipulation, motion planning, and automation without requiring physical hardware. Robotic arm simulators are widely used in industrial automation, manufacturing, assembly lines, and educational robotics.

Introduction

Robotic arms are programmable mechanical devices designed to perform tasks such as pick-and-place operations, welding, painting, and assembly. A 6-axis robotic arm provides six degrees of freedom, allowing flexible movement in three-dimensional space.

The main objective of this project is to simulate robotic arm motion and understand the concepts of forward kinematics, inverse kinematics, and trajectory planning.

Objectives

- Design a virtual 6-axis robotic arm.
- Implement inverse kinematics algorithms.
- Move the robotic arm to target coordinates.
- Generate smooth motion trajectories.
- Visualize robotic movement in simulation.
- Study robotic automation concepts.

Problem Statement

Industries require robotic systems capable of precise and repeatable movements. Developing and testing these systems on real hardware can be expensive. A simulation-based robotic arm allows learning and experimentation at low cost while providing an understanding of robotic control systems.

Software and Hardware Requirements

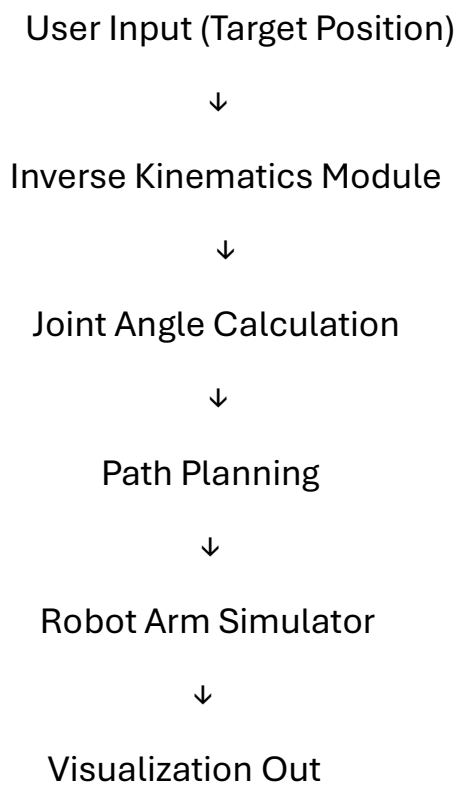
Hardware

- Laptop/Desktop
- Minimum 4 GB RAM
- Intel i3 or higher processor

Software

- Python 3.x
- NumPy
- Matplotlib
- ROS (Optional)
- Gazebo Simulator
- VS Code

System Architecture



Methodology

Step 1: Define Robot Structure

The robotic arm consists of:

- Base Joint
- Shoulder Joint
- Elbow Joint
- Wrist Pitch
- Wrist Roll
- End Effector

Step 2: Input Target Position

The user enters target coordinates:

(x, y, z)

Example:

$(0.3, 0.2, 0.4)$

Step 3: Inverse Kinematics

The inverse kinematics algorithm calculates the joint angles required to reach the target position.

Step 4: Path Planning

The system generates a collision-free path from the current position to the target position.

Step 5: Simulation

The robotic arm moves smoothly toward the target point and displays the motion graphically.

Algorithm

1. Start Program
2. Define Robot Parameters
3. Input Target Position
4. Compute Inverse Kinematics
5. Calculate Joint Angles
6. Generate Trajectory
7. Move Arm in Simulation

8. Display Results

9. Stop Program

Sample Python Code

```
import math  
  
x = 0.3  
  
y = 0.2  
  
theta = math.atan2(y, x)  
  
print("Base Joint Angle:", math.degrees(theta))
```

Expected Results

- Successful movement of robotic arm.
- Accurate target point positioning.
- Smooth trajectory generation.
- Visual simulation of arm motion.

Applications

- Industrial Automation
- Pick and Place Systems
- Medical Robotics
- Assembly Lines
- Educational Robotics
- Research Laboratories

Advantages

- Low-cost development.
- Safe testing environment.
- Easy debugging.
- No hardware required.
- Better understanding of robotics concepts.

Future Scope

- Object detection using Computer Vision.
- AI-based path planning.
- Obstacle avoidance.
- Real robotic arm implementation.
- ROS2 integration.
- Machine learning control systems.

Conclusion

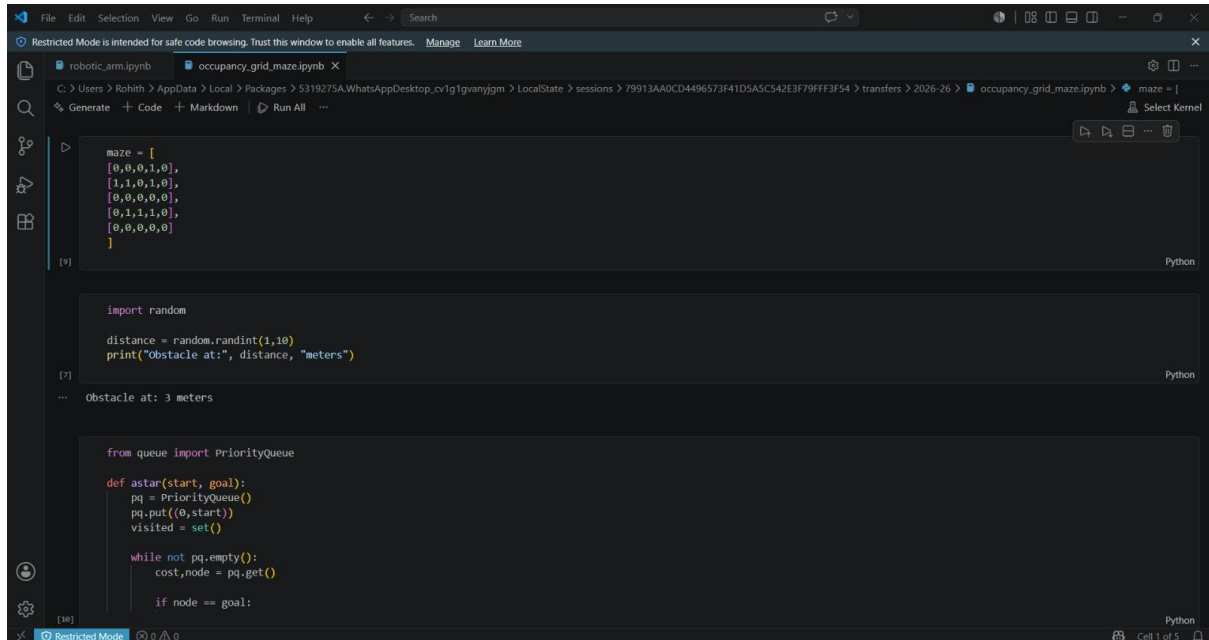
The 6-Axis Robotic Arm Simulator successfully demonstrates robotic motion using inverse kinematics and path planning techniques. The project provides practical knowledge of robotics, automation, and simulation technologies. It serves as a strong foundation for advanced robotic applications in industrial and research environments.

References

1. ROS Documentation
2. Gazebo Documentation

3. MoveIt Motion Planning Framework

4. Research papers on robotic arm kinematics and simulation.



```
File Edit Selection View Go Run Terminal Help
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

robotic_arm.ipynb occupancy_grid_maze.ipynb X
C:\Users\Rohith> AppData\Local\Packages> 5319275A.WhatsAppDesktop_cv1g1gvanyjgm> LocalState> sessions> 79913AA0CD4496573F41D5A5C542E3F79FF3F54> transfers> 2026-26> occupancy_grid_maze.ipynb> maze = [
Generate + Code + Markdown Run All
Select Kernel

maze = [
[0,0,0,1,0],
[1,1,0,1,0],
[0,0,0,0,0],
[0,1,1,1,0],
[0,0,0,0,0]
]

[9] Python

import random

distance = random.randint(1,10)
print("Obstacle at:", distance, "meters")

[7] Python

Obstacle at: 3 meters

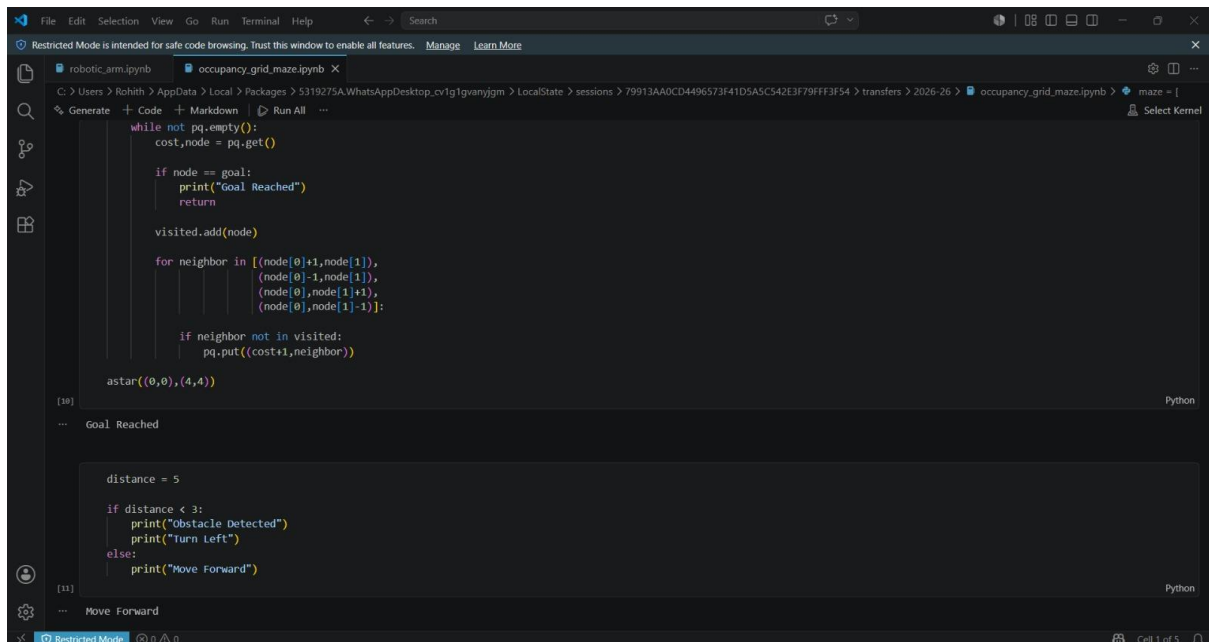
from queue import PriorityQueue

def astar(start, goal):
    pq = PriorityQueue()
    pq.put((0,start))
    visited = set()

    while not pq.empty():
        cost,node = pq.get()

        if node == goal:
            return node

[38] Python
```



```
File Edit Selection View Go Run Terminal Help
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

robotic_arm.ipynb occupancy_grid_maze.ipynb X
C:\Users\Rohith> AppData\Local\Packages> 5319275A.WhatsAppDesktop_cv1g1gvanyjgm> LocalState> sessions> 79913AA0CD4496573F41D5A5C542E3F79FF3F54> transfers> 2026-26> occupancy_grid_maze.ipynb> maze = [
Generate + Code + Markdown Run All
Select Kernel

while not pq.empty():
    cost,node = pq.get()

    if node == goal:
        print("Goal Reached")
        return node

    visited.add(node)

    for neighbor in [(node[0]+1,node[1]),
                    (node[0]-1,node[1]),
                    (node[0],node[1]+1),
                    (node[0],node[1]-1)]:

        if neighbor not in visited:
            pq.put((cost+1,neighbor))

    astar((0,0),(4,4))

[38] Python

Goal Reached

distance = 5

if distance < 3:
    print("Obstacle Detected")
    print("Turn Left")
else:
    print("Move Forward")

[11] Python

Move Forward

Cell 1 of 5
```